Fundamental Programming in JAVA

ARRAY OF OBJECTS

Objectives

- To Implement a type that uses an array instance variable.
- To describe how to create arrays of objects.
- To store and process objects in arrays.
- To program common array of objects operations (displaying arrays, summing all elements and finding the minimum and maximum).

Array Of Objects

- An **array** is a collection of the same data type that dynamically creates objects and can have elements of primitive types.
- **Java** is an object-oriented programming language, and most of the work is done with the help of objects.

Array Of Objects

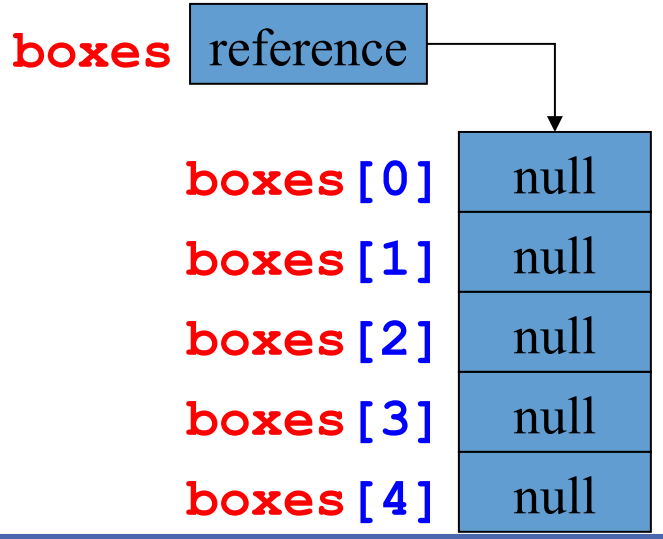
- Java allows us to store objects in an array.
- If you want to store a single object in your program, then you can do so with the help of a variable of type object. But when you are dealing with numerous objects, it is advisable to use an array of objects.

Array Of Objects

- An array of objects is an array of **references** to **objects**

```
Rectangle[] boxes = new Rectangle[5];
```

- The **boxes** variable holds the address (reference) of a **Rectangle** array.



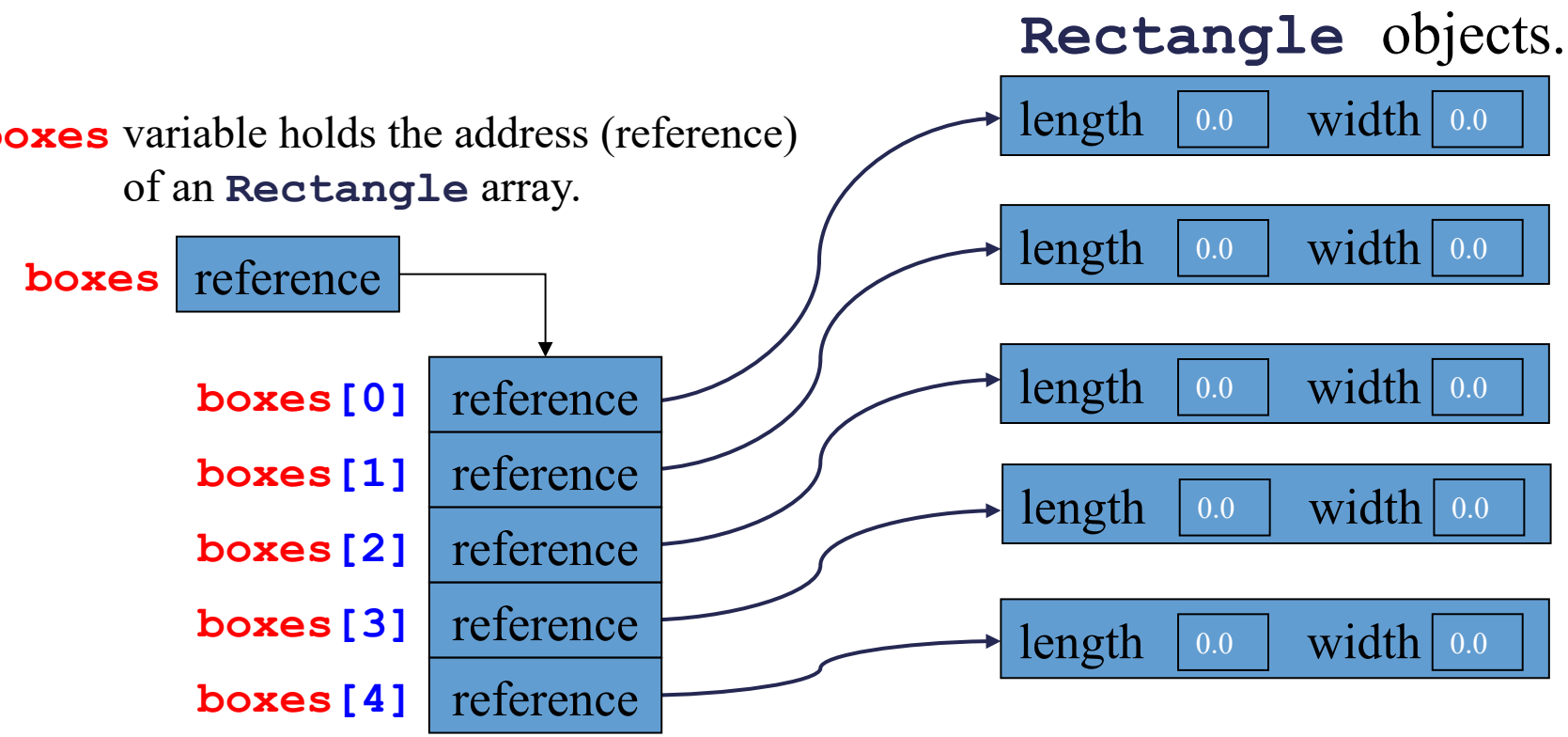
The array is an array of references to **Rectangle** objects.

Instantiating Arrays of Objects

- Each element needs to be initialized.

```
for (int i = 0; i < boxes.length; i++)
     boxes[i] = new Rectangle();
```

The **boxes** variable holds the address (reference) of an **Rectangle** array.



Invoking a method in array of objects

- An array of objects is actually an array of reference variables.

  So invoking `boxes[3].setLength(8.3)` involves **two** levels of referencing as shown in the next figure.

```
boxes[3].setLength(8.3)
```

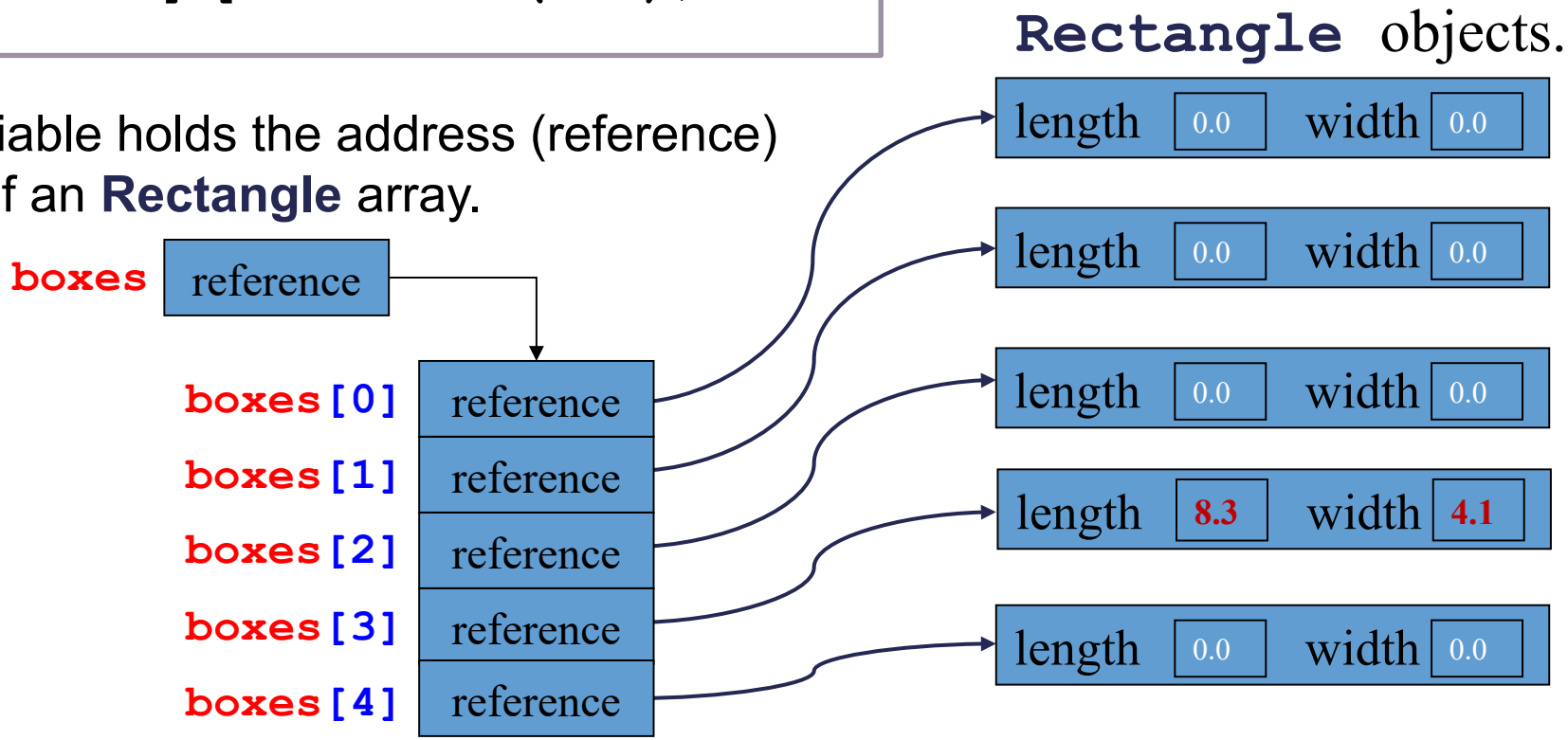
`boxes` references to the **entire array**.

`boxes[3]` references to a **Rectangle object**.

Invoking a method in array of objects(cont)

```
boxes[3].setLength(8.3);
boxes[3].setWidth(4.1);
```

The **boxes** variable holds the address (reference) of an **Rectangle** array.



Array Of Objects - Example

```
public class Rectangle {
    private double length;
    private double width;

     public Rectangle(double length, double width)  {
          this.length = length;
          this.width = width;          }

    public void setLength(double L)
    {     length=L;       }

    public void setWidth(double W)
    {     width=W;         }

    public double getWidth()
    {    return width;       }

    public double getLength()
    {    return length;      }

    public double getArea()
    {     return length*width;    }

}
```

Array Of Objects – Example Cont.

```java
import java.util.Scanner;
public class ArrayOfObjects
{
   public static void main(String[] args)
   {  // Create an array to hold rectangle's objects data.
      Rectangle[] rectarray = new Rectangle[5];
      double length; //to hold length of rectangle
      double width;  //to hold width of rectangle

      // Create a Scanner object for keyboard input.
      Scanner console = new Scanner(System.in);

      for (int i = 0; i < 3; i++)
      {  System.out.print("Enter the length of Rectangle " +(i + 1)+ ": ");
         length = console.nextDouble();
         System.out.print("Enter the width of Rectangle " + (i + 1)+ ": ");
         width = console.nextDouble();
         rectarray[i] = new Rectangle(length, width);
         System.out.println();
      }

      for (int i = 0; i < 3; i++)
      {
         System.out.println("Area of Rectangle " + (i + 1) + ": "
                                 + rectarray[i].getArea());
      }
   }
}
```

Array Of Objects - Example

Car.java

```java
public class Car {
    String type;
    double price;

    public Car(String type, double price) {
        this.type = type;
        this.price = price;
    }

    public Car() {
    }

}
```

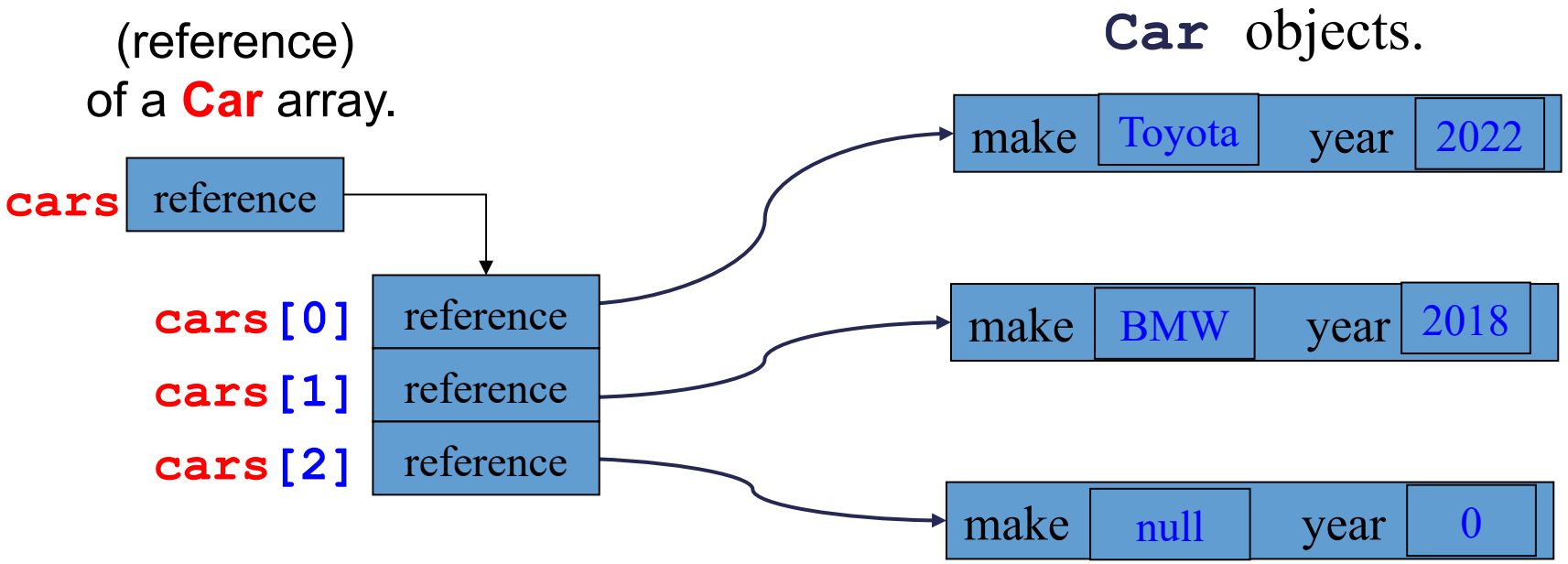
javaapplication1.Car

- initializing each member of the array when creating it.

```java
Car[] cars = new Car[]{
    new Car("Toyota", 2022),
    new Car("BMW", 2018),
    new Car()};
```

Array Of Objects - Example

The **cars** variable holds the address (reference) of a **Car** array.



An Array Instance Variable

- As you continue your study of computing fundamentals, you will spend a fair amount of time using arrays and managing collections of data. The Java array is one of several data storage structures used inside classes with the main task of storing a **collection**. These are known as **collection** classes with some of the following characteristics:
  - The main responsibility of a collection class is to store a collection of objects
  - Objects are added and removed from a collection
  - A collection class allows clients to access the individual elements
  - A collection class may have search-and-sort operations for locating a particular item.
  - Some collections allow duplicate elements; other collections do not

An Array Instance Variable-Example

Student.java | Department.java | DepartmentTest.java

Source | History

```java
public class Student {
    String name;
    int id;
    public Student() {
        name="No Name";
        id=-1;
    }
    public Student(String name, int id) {
        this.name = name;
        this.id = id;
    }
    @Override
    public String toString() {
        return "Student{" + "name=" + name +
                ", id=" + id + '}';
    }
}
```

An Array Instance Variable-Example

Student.java | Department.java | DepartmentTest.java

Source | History

```
4
5   public class Department {
6
    private String name;
    private Student [] studentsList;
9   private int no_of_students;
10  public Department(String name)
11  {
12      this.name = name;
13      studentsList = new Student[10]; //Array if Objects
14      no_of_students=0;
15  }
16
17
18
```

Data Fields

Constructor

An Array Instance Variable-Example

Student.java | Department.java | DepartmentTest.java

Source | History

```java
public void printStudents()
{
    System.out.print("The Department " +  name +
            " has the following students : \n");
    for(int i =0;i<no_of_students;i++)
    {
      System.out.println(studentsList[i].toString());
    }
}

public void addStudent(Student s)
{
    studentsList[no_of_students] = s;
    no_of_students++;
}
```

This method prints the data of all students in the Department

This is how to add a new student (Object) to the department

An Array Instance Variable-Example

Student.java | Department.java | DepartmentTest.java

Source | History

```
47
48
49      public boolean deleteStudent(Student s)
50      {
51      for(int i =0;i<no_of_students;i++)
52          {
53            if(studentsList[i].id==s.id)
54            {
55              studentsList[i]=null;
56              no_of_students--;
57                return true;          }
58          }
59      return false;
60      }
61
```

This is how to delete a student (Object) form the department

An Array Instance Variable-Example

Student.java | Department.java | DepartmentTest.java

Source | History

```
62
63
64    public int Search(Student s)
65        {
66        for(int i =0;i<no_of_students;i++)
67            {
68              if(studentsList[i].id==s.id)
69              {
70               return i;
71              }
72            }
73        return -1;
74        }
75    }
76
```

This method searches for a specific student(Object), returns his position in the array, or returns -1 if it does not exist

An Array Instance Variable-Example

Student.java | Department.java | DepartmentTest.java

Source | History

```java
5   public class DepartmentTest {
6       public static void main(String[] args) {
7       Department dept1 = new Department("Computer Science");
8           Student st1 = new Student("Sami Ahmad",2021);
9           Student st2 = new Student("Hani",2022);
10          Student st3 = new Student("Muna",2023);
11          dept1.addStudent(st1);dept1.addStudent(st2);
12          dept1.addStudent(st3);
13          dept1.printStudents();
14          int R=dept1.Search(st3);
15          if(R==-1)
16              System.out.println("Student Not Found");
17          else
18              System.out.println("Found at index : "+R);
19          dept1.deleteStudent(st3);
20          dept1.printStudents();
21      }
```

Student.java | DepartmentTest.java | Output - JavaApplication1 (run)

```
run:
The Department Computer Science has the following students :
Student{name=Sami Ahmad, id=2021}
Student{name=Hani, id=2022}
Student{name=Muna, id=2023}
Found at index : 2
The Department Computer Science has the following students :
Student{name=Sami Ahmad, id=2021}
Student{name=Hani, id=2022}
BUILD SUCCESSFUL (total time: 6 seconds)
```

References

- Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes: edugate.asu.edu.jo.
- lecture Videos: YouTube channel.
- How to install JDK & NetBeans IDE manual.